

Spark SQL

DataFrames and Databases

DataFrame & Database table: conceptually equivalent.

Spark makes them interoperable

DataFrames & SQL

- Query existing Spark DataFrames (however created) with SQL
 - Same functionality, different interface
 - Legacy SQL

Read data and persistency

- Load data from Hive / other databases
- Save tables to Hive

Explore the Yelp dataset

```
yelp_df = sqlCtx.load(  
    source='com.databricks.spark.csv',  
    header = 'true',  
    inferSchema = 'true',  
    path =  
    'file:///usr/lib/hue/apps/search/examples/collections/solr_co  
nfigs_yelp_demo/index_data.csv')
```

Register as a SQL table

- DataFrame already has Schema
- Create a temporary table with:
`yelp_df.registerTempTable("yelp")`

<https://spark.apache.org/docs/3.1.1/api/python/reference/api/pyspark.sql.DataFrame.html>
<https://spark.apache.org/docs/latest/sql-ref.html>

Run SQL statements

```
filtered_yelp = sqlCtx.sql("SELECT * FROM yelp WHERE  
useful >= 1")
```

```
filtered_yelp
```

```
Out[]: DataFrame[business_id: string, cool: int, date: string,  
funny: int, id: string, stars: int, text: string, type: string,  
useful: int, user_id: string, name: string, full_address:  
string, latitude: double, longitude: double, neighborhoods:  
string, open: string, review_count: int, state: string]
```

Filtering

```
filtered_yelp.count()
```

```
Out[]: 601L
```

```
yelp_df.filter(yelp_df.useful >= 1).count()
```

```
Out[]: 601L
```

aggregation

```
sqlCtx.sql("SELECT MAX(useful) AS max_useful FROM  
yelp").collect()
```

```
Out[]: [Row(max_useful)=28)]
```

```
yelp_df.agg({"useful": "max"}).collect()
```

```
Out[]: [Row(MAX(useful#267)=28)]
```

Join - select - show

```
useful_perc_data.join(  
    yelp_df,  
    yelp_df.id == useful_perc_data.uid,  
    "inner"  
).select(useful_perc_data.uid, "useful_perc", "review_count")  
|  
|
```

Register as SQL table

```
useful_perc_data.registerTempTable("useful_perc_data")
```

join

```
sqlCtx.sql(  
  ""SELECT useful_perc_data.uid, useful_perc,  
  review_count  
  FROM useful_perc_data  
  INNER JOIN yelp  
  ON useful_perc_data.uid=yelp.id""  
)
```

Performance

- Either DataFrame calls or SQL
- Same under-the-hood optimizer (Catalyst)
- Creates DAG
- Parallel execution
- Creates bytecode

Spark Dataframes: Catalyst Optimizer

- Spark DataFrames use a **catalyst optimizer** to optimize query performance.
- The catalyst optimizer is responsible for finding the most efficient ways to apply transformations and actions to improve query performance.
- Catalyst optimizer makes DataFrames perform better than RDDs.
- Queries undergo planning stages before executing as a physical plan.
- Using DataFrames relies on catalyst optimizer for query optimization instead of using RDDs.

Example

For example, you are trying to calculate the average salary of employees by age

using RDDs

```
fileRdd = sc.textFile("/employeeData.csv")
```

```
fileRdd.map( x => {val fields = x.split(",") \
    (fields(1), fields(2)) }) \
    .map(x => (x._1, (x._2, 1))) \
    .reduceByKey((x,y) => (x._1 + y._1, x._2 + y._2)) \
    .map( x => (x._1, x._2._1 / x._2._2))
```

using Dataframes

```
data_df = fileRdd.toDF("username", "age", "salary")
data_df.groupBy($"age").agg(avg("salary"))
```

Difference in RDD and Dataframe Approach

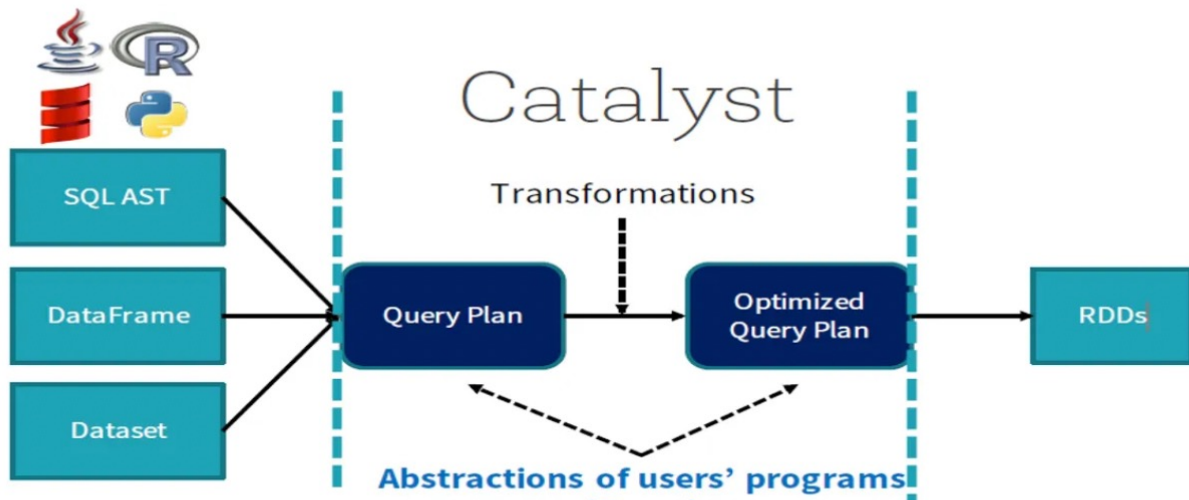
In RDD approach:

We are telling Spark exactly how to do using Lambda functions, and Spark can't optimize that. It directly sends those functions to executors to work on the data. If there's any possible optimization, we have to do it ourselves.

In DataFrame approach:

We are using the declarative way in which we are telling Spark what to do and leaving the how to do part on Spark's optimizer. This makes DataFrames' optimization possible via catalyst optimizer.

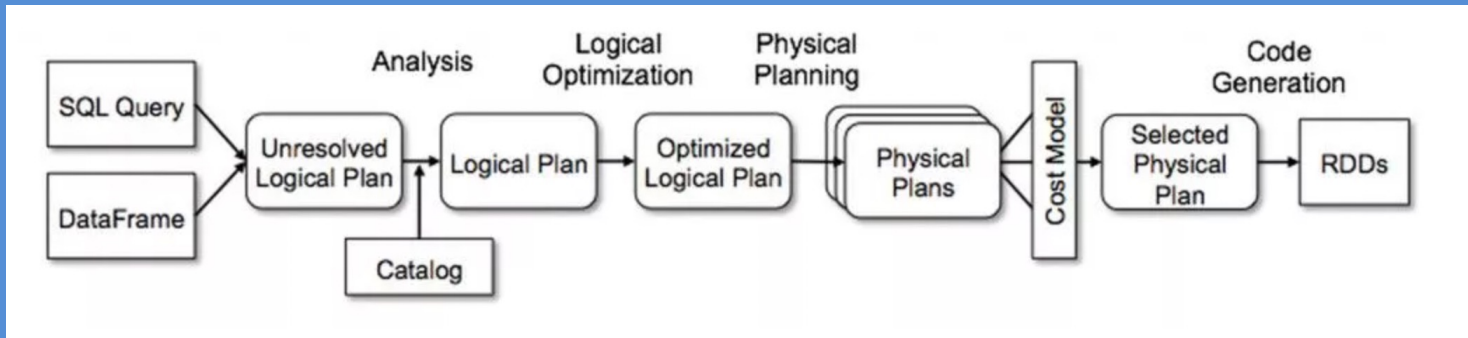
Catalyst Optimizer



- https://medium.com/@Shkha_24/catalyst-optimizer-the-power-of-spark-sql-cad8af46097f

Catalyst Optimizer Working

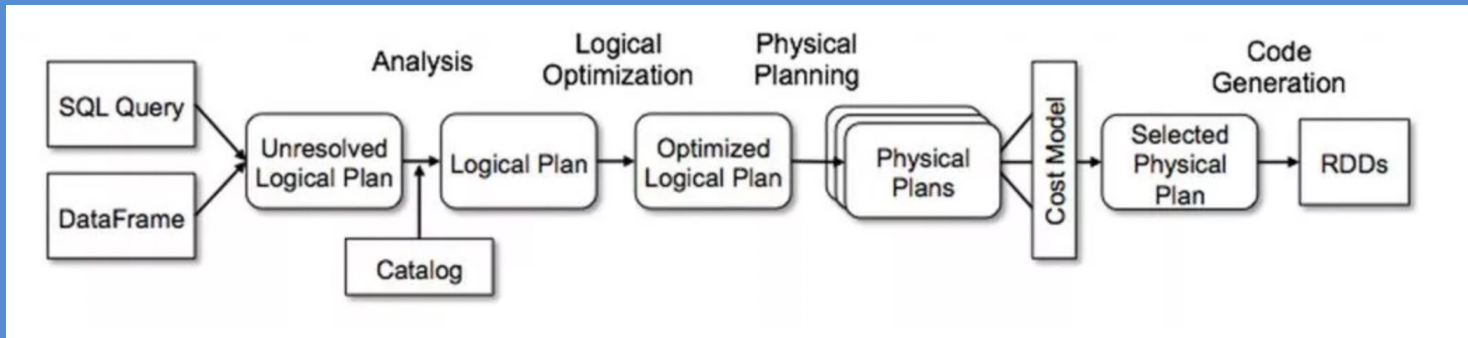
- It creates an **unresolved logical plan** and checks for the validity of column names and table names etc.
- After that, a **resolved logical plan** is created. At this step, commands might get re-organized to optimize performance.
- Catalyst optimizer might generate at least one **physical plan** at this stage to execute the query. This stage represents what Spark will actually do after optimizations have been applied.
- In the case of more than one physical plan, the cost for each plan is evaluated using a **cost model**. Plan with the **best performance** is selected, compiled to java bytecode, and executed.



Catalyst Optimizer Working

Logical plan: Logical plan describes computation on the data without defining how to conduct the computation.

Physical plan: Physical plan describes computation on the data with specific definitions and how to conduct the computation



Resources

- [https://medium.com/@Shkha_24/catalyst-optimizer-the-power-of-spark-sql-cad8af46097f\](https://medium.com/@Shkha_24/catalyst-optimizer-the-power-of-spark-sql-cad8af46097f)
- <https://www.systemsltd.com/blogs/apache-spark-architecture-and-application-lifecycle>